Combining the Formal and the Informal

Jonathan Reicher Shachar Itzhaky

June 2, 2026

Problem Statement

In the intersection between the worlds of mathematical theory, formal logic, and programming, exists a species of tools called proof assistants. These are programming languages, meant not for building software, but for building mathematical proofs. Proof assistants, contrary to pen and paper, check formalized proofs for correctness and soundness.

These so called proof assistants admit only entirely formalized proofs, where every step is fully justified based on a complex logical semantics sitting on top of a complex declarative semantics. That is to say, not only are the logical steps themselves complex and rigorous, but so is the way you express your proof and how the steps are applied.

For many years, proof assistants were a niche, unpopular family of tools, used mostly by experts and enthusiasts. However, one member of that family in particular has been gaining traction in the mathematical community, bringing many inexperienced users to the world of interactive theorem proving.

Lean (TODO: cite?) is an interactive proof assistant geared towards a general math audience. Two things that make it especially appealing are its large ecosystem and its intuitive syntax. For example, it has a large library of formalized mathematics called `Mathlib`, which contains many foundational and advanced mathematical results and theories, and includes notation familiar to mathematicians. Despite that, there are some challenging aspects of adopting Lean, and one in particular is how complex its semantics and type system are.

Existing Solutions

When first wanting to formalize a proof in Lean, one usually takes human mathematicians and sends them to learn the language, and write their proof. This solution comes with the obvious drawback of being slow,

requiring a lot of human time and effort. Lean has a steep learning curve, and it can take months to learn the basics.

A more automated approach is to formalize just the theorems and definitions, and then use an automated theorem prover to fill in the gaps. Several such tools exist, such as Aesop, Grind, and LeanEgg (TODO: cite..?). These tools complete a theorem, or part of a theorem, using various heuristics and algorithms. They tend to be integrated into Lean, and more fit to use as part of interactive proof development, rather than full automation. Moreover, they tend to work only on simple theorems, fail without human guidance, and even when they succeed, they do not produce readable a proof.

For a more wholistic approach for formalizing a proof, one can reach for any off-the-shelf generative large language model such as ChatGPT. In an agentic setting, these systems have gotten good at generating idiomatic and correct Lean code. One can write their paper, provide it to the agent, and prompt it to generate a formal Lean proof. One can even interact with it to steer towards better directions. There are even tools in place to help large language models work with Lean such Aristotle (TODO: cite?). These systems suffer a number of drawbacks. To name a few: They lack reproducibility, are usually based on cloud services, and are not fit with an interactive and iterative workflow.

There are also libraries for making proofs easier to read and write. One example is Waterproof. This is a library for the Rocq proof assistant, which adds notation inspired by pen-and-paper proofs. An example proof, written with and without Waterproof respectively, can be seen in figures 1 and 2.

It does require the proof to be fully formalized, and informal proofs would require major work to be adapted to Waterproof.

TODO: Mention open ai's paper that adi sent

Our Approach

We define a language for writing *informal* proofs that can be partially translated to Lean, to help with starting a formal proof. The language is a modification of markdown, with some syntax constructs being recognized as formal. All parts are readable as they are written. The language can be interfaced via Lean directly by a macro, and can access Lean and `Mathlib` symbols. It is meant as a way to write an informal proof to be understood, and a formal proof to be checked.

This workflow is in accordance to how researchers tend to use Lean, as a formal proof in Lean often comes as a supplementary part of a paper proof inside a research paper.

```
Goal 2 is the infimum of [2, 5).
Proof.
We need to show that (2 is a _lower bound_ for [2, 5)
  ∧ (∀ l ∈ ℝ, l is a _lower bound_ for [2, 5) ⇨ l  2)).
We show both statements.
  - We need to show that (2 is a lower bound for [2, 5)).
    We need to show that (∀ c ∈ [2, 5), 2 ≤ c).
    Take c ∈ [2, 5).
    We conclude that (2 ≤ c).
  - We need to show that
    (∀ l ∈ ℝ, l is a lower bound for [2, 5) ⇒ l ≤ 2).
    Take l ∈ ℝ. Assume that (l is a lower bound for [2, 5)).
    We conclude that (l ≤ 2).
Qed.
```

Figure 1: A proof written in Rocq with Waterproof.

```
Goal is_infimum 2 (interval_half_open 2 5).
Proof.
  unfold is_infimum.
  split.
  - (* We need to show that 2 is a lower bound for [2, 5) *)
    unfold lower_bound.
    intros c Hc.
    destruct Hc as [H2 _].
    exact H2.
  - (* We need to show that any lower bound l is <= 2 *)
    intros l Hl.
    unfold lower_bound in Hl.
    apply Hl.
    (* Concluding l <= 2 by applying the lower bound to 2 ∈ [2,
5) *)
    split.
    + lra.
    + lra.
Qed.
```

Figure 2: The same proof, written in Rocq without Waterproof.

#	**Assumption**	**Explanation**
1	$H = \{\{\langle x, y\rangle, \langle x+1, y\rangle\} \mid x \in 0..8, y \in 0..9\}$	Set of possible horizontal tiles
2	$V = \{\{\langle x, y\rangle, \langle x, y+1\rangle\} \mid x \in 0..9, y \in 0..8\}$	Set of possible vertical tiles
3	$D \subseteq H \cup V$ is a valid tiling	
4	$\lvert D \cap H\rvert = \lvert D \cap V\rvert$	Assume for contradiction
	Proposition	**Explanation**
1	$\lvert D\rvert = 50$	Our board's size is 100, and every element is a set of size 2, hence: 50 tiles
2	$(D \cap H) \cup (D \cap V) = D$	By $D \subseteq H \cup V$
3	$(D \cap H) \cap (D \cap V) = \emptyset$	Because H and V are disjoint
4	$\lvert D \cap H\rvert = \lvert D \cap V\rvert = 25$	By the assumption, and by previous two conjectures
5	$\sum_{t \in D \cap V} \sum_{\langle x,y\rangle \in t} x$ is even	Every number in the sum appears twice, as the x's of a tile's cells are equal
6	$\sum_{t \in D \cap H} \sum_{\langle x,y\rangle \in t} x$ is odd	Here every number appears with its successor by similar reasoning
7	$\sum_{t \in D} \sum_{\langle x,y\rangle \in t} x$ is even	By calculation
8	$\sum_{t \in D} \sum_{\langle x,y\rangle \in t} x = \sum_{t \in D \cap V} \sum_{\langle x,y\rangle \in t} x + \sum_{t \in D \cap H} \sum_{\langle x,y\rangle \in t} x$	By $(D \cap H) \cup (D \cap V) = D$ and $(D \cap H) \cap (D \cap V) = \emptyset$
9	Contradiction	In the previous proposition, the left-hand side is even while the right-hand side is odd

Figure 3: A proof sketch of the problem.

Example For example, for the following markdown document:

```
Let $H = \{ \{ (x, y), (x + 1, y) \} \mid x \in 0..8, y \in
0..9 \} \}$ be the set of horizontal tiles on the board.
```

The tool generates the following LEAN code:

```
def H : Set (Fin 10 × Fin 10) := { (x, y) | (x ∈ Finset.range 9) (y
∈ Finset.range 10) }
```

The tool is invoked by simply calling the macro `define_latex file "foo.md"`, with `foo.md` replaced with the file name of the markdown document of your choice. It is also possible to use `define_latex "Hello` $x = world$`"`.

Preliminary Work

As a case study, we started by taking a look at the following problem:

> Can you cover a 10×10 board with 1×2 tiles, such that the number of horizontal and vertical tiles is the same?

As it turns out, you can't. One can prove that it is impossible. A short intuitive proof can be seen in figure 3. While this proof is readable and

correct, and even relatively formal, it does not translate well to Lean. As an example, here is a part of the proof that represents conjecture 3.

```
lemma disjoint_hvtiles (tiling : DominoTiling n m) :
  Disjoint (htiles tiling) (vtiles tiling) := by
  unfold htiles vtiles
  apply Finset.disjoint_filter_filter'
  apply Pi.disjoint_iff.mpr
  intros
  unfold horizontal vertical
  split <;> simp
```

One of the challenges in translating a proof to Lean is representation. It is often much more idiomatic to use one of many possible representations. For something that might fit as a set for a mathematician, in Lean it might be better use the `Set` type, or the `Finset`, or maybe represent it as a type, or even a predicate. The most idiomatic choice for representation depends on use.

As a proof of concept, we made a tool which compiles inline math in a markdown document to Lean code, and uses a static analysis of the document to determine which representation to use for each mathematical object. You can invoke the tool interactively inside a Lean document, giving it a markdown document, and it declares variables and definitions which can be used from that point on as if they were declared in the file itself directly.

The tool we have right now is very basic, and the only possible representations are `Set` or `Finset` for sets, and `Multiset` for multisets and `Prod` for tuples.

References